OS Assignment 2 question # 1

OS Assignment #2

Abdullah Sonija 24k-0013

Q1 Chapter 4

1. A web server that handles each client request in a seprate thread, allowing simultaneous connection.
(ii) A word processor where one thread responds to user keystrokes while another thread performs background spell-checking or document saving.
(iii) An image rendering application where the image is divided into segments, and each segment is processed concurrently by different threads.

2. Formula: $\text{Speedup} = \frac{1}{(1-P) + P/N}$; $P = 0.60$

(a) $N = 2$: $\frac{1}{1-0.6+0.6/2} = 1.43$

(b) $N = 4$: $\frac{1}{1-0.6+0.6/4} = 1.82$

3. Task parallelism because each thread is managing a unique network connection & executing the server's logic independently for different users.

4. User level thread are managed by a thread library where as kernel level threads are managed directly by the performs a blocking the entire gets blocked where as in kernel level if one thread blocks, the kernel can schedule another thread

User-level threads are better when for performance in critical.
kernel-level threads are better when multiprocessing is available & frequent I/O is envolved

5. ~~It requires allocating~~
(i) Interept or system call
(ii) The context of current thread is saved
(iii) Switch to kernel mode
(iv) The thead in ready queue is restored
(v) Context of new thread is restored
(vi) PC is updated and resumed

6.) Requires allocating a thread Control block, a thread identifier, a program counter, a register set & a stack for the Thread Creation. where as inProcess creation, memory allocation for code, data & heap segment (heavy work), along with file descriptors, & a full process Block.

7. Yes. it is neccessary. Real time process requires require strict, guranteed execution timelines, If it is not bound to a specific LWP, it might have to wait for an LWP to become available before it can be scheduled by the kernel.

8. 1. A strictly sequential Mathematically Computation like fibonacci sequence
2. A simple shell script that executes a single quick command. The overhead of creating & joining threads would take longer than the task itself.

9. When the application involves blocking i/o operations.

10. Heap memory & global variables.

11. Generally it can not perform better, user-level theards are managed by a library in user space entirely. which the Os is unaware, To the Os ~~level~~ kernel, the entire process looks like a single ~~proce~~ thread of execution

12. No, thread share the same memory space. If chrome uses thread, one tab experiencing a fatal error would crash the entire browser. Using seperate process ensures memory isolation & stability.

13. Yes, concurrency means multiple tasks making progress at the same time.
Parallelism means tasks are executing exactly at the identical moment. A single core system multitasking is concurrent but not parallel.

14. (a) $P = 0.4$; $n = 8$: $1/(0.6 + 0.4/8) = 1.53$
(b) $P = 0.67$; $n = 2$: $1/(0.33 + 0.67/2) \Rightarrow 1.50$
(b) $P = 0.4$; $n = 16$; $1/(0.6 + 0.4/16) = 1.60$
(b) $P = 0.67$, $n = 4$: $1/(0.33 + 0.67/4) = 2.01$.
(a) $P = 0.9$, $n = 4$; $1/(0.1 + 0.9/4) = 3.07$
(b) $P = 0.9$, $n = 8$; $1/(0.1 + 0.9/8) = 4.70$.

15. (i) Photos: Data Parallelism
(ii) Matrix : Data Parallelism
(iii) Network: Task Parallelism
(iv) Fork join: Data Parallelism
(v) Central Dispatch : Both

16. Input /Output : 1 thread , since all input /output goes into a single file.

CPU-intensive portion : 4 threads, since System has 2 dual core processors, and uses a one-to one threading model

17. Process : 6
Threads : 8 (P_1 & P_2 has an additional one)

18. Windows strictly seperate processes (resource containers) from threads. In contrast, the linux treats both equally as 'task' created by the clone() systemcall, differentiating them only by the level of resources they are flagged to share.

19. Line C : 5
Line P : 0 since fork() creates completely separate memory space for child

4.22 Write a multithreaded program that calculates various statistical values for a list of numbers. This program will be passed a series of numbers on the command line and will then create three separate worker threads. One thread will determine the average of the numbers, the second will determine the maximum value, and the third will determine the minimum value.
For example, suppose your program is passed the integers 90 81 78 95 79 72 85
The program will report

The average value is 82
The minimum value is 72
The maximum value is 95

The variables representing the average, minimum, and maximum values will be stored globally. The worker threads will set these values, and the parent thread will output the values once the workers have exited. (We could expand this program by creating additional threads that determine other statistical values, such as median and standard deviation.)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int average = 0;
int maximum = 0;
int minimum = 0;

typedef struct {
    int *arr;
    int size;
} ThreadData;

void *calculate_average(void *param) {
    ThreadData *data = (ThreadData *)param;
    int sum = 0;
    for (int i = 0; i < data->size; i++) {
        sum += data->arr[i];
    }
    average = sum / data->size;
    pthread_exit(0);
}

void *calculate_maximum(void *param) {
    ThreadData *data = (ThreadData *)param;
    maximum = data->arr[0];
    for (int i = 1; i < data->size; i++) {
        if (data->arr[i] > maximum) {
            maximum = data->arr[i];
        }
    }
```

```
    pthread_exit(0);
}

void *calculate_minimum(void *param) {
    ThreadData *data = (ThreadData *)param;
    minimum = data->arr[0];
    for (int i = 1; i < data->size; i++) {
        if (data->arr[i] < minimum) {
            minimum = data->arr[i];
        }
    }
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: ./stats <num1> <num2> ... <numN>\n");
        return -1;
    }
    int size = argc - 1;
    int *numbers = (int *)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        numbers[i] = atoi(argv[i + 1]);
    }
    pthread_t tid1, tid2, tid3;
    ThreadData data = {numbers, size};
    pthread_create(&tid1, NULL, calculate_average, &data);
    pthread_create(&tid2, NULL, calculate_maximum, &data);
    pthread_create(&tid3, NULL, calculate_minimum, &data);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    pthread_join(tid3, NULL);
    printf("The average value is %d\n", average);
    printf("The minimum value is %d\n", minimum);
    printf("The maximum value is %d\n", maximum);
    free(numbers);
    return 0;
}
```

4.23 Write a multithreaded program that outputs prime numbers. This program should work as follows: The user will run the program and will enter a number on the command line. The program will then create a separate thread that outputs all the prime numbers less than or equal to the number entered by the user.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *print_primes(void *param) {
    int max_num = atoi((char *)param);
    printf("Prime numbers up to %d:\n", max_num);
    for (int i = 2; i <= max_num; i++) {
        int is_prime = 1;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                is_prime = 0;
                break;
            }
        }
        if (is_prime) {
            printf("%d ", i);
        }
    }
    printf("\n");
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: ./primes <number>\n");
        return -1;
    }
    if (atoi(argv[1]) < 2) {
        printf("Please enter a number >= 2\n");
        return -1;
    }
    pthread_t tid;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_create(&tid, &attr, print_primes, argv[1]);
    pthread_join(tid, NULL);
    return 0;
}
```